Neel Hand-off

🔁 TrustX ATAP – Engineering Documentation Handoff Template

> **Owner:** Neel

> **Last Updated:** June 22, 2025

> **Reviewed By:** To be Reviewed by Rama

> **Module(s):** AWS Infrastructure, Profile-360 Pipeline, Automated Control Testing, Multi-tenant Architecture, AppRunner Services, Frontend Controls UI, Document Editor, Integrations, Lambda Functions

---

📋 1. Summary of Work

Modules Owned:

- AWS Infrastructure & CDK Stacks
- Profile-360 Data Pipeline (Lambda Functions & Step Functions)
- Automated Control Testing System
- AppRunner Services for AI Query Processing
- Integration Services (GitHub, JIRA, EntraID)
- Frontend Controls UI & Compliance Interface
- Document Editor & Management System
- Policy Management Interface
- Lambda Functions Suite (Documents, List, Configure, Delete, Sync, Scheduler)

Brief Overview:

> Neel has helped in implement a comprehensive AWS-based infrastructure for TrustX ATAP, including:
>
> - **Profile-360 Pipeline:** Multi-tenant data processing pipeline that collects, formats, and ingests data from GitHub, JIRA, and EntraID into Neo4j databases for AI-powered compliance analysis
> - **Automated Control Testing:** End-to-end system that automatically tests compliance controls (like TX-MON04.01) by collecting AWS infrastructure data, integration data, and document embeddings, then using LLM analysis to generate compliance scores and evidence
> - **Multi-tenant Architecture:** Strict tenant isolation with dynamic tenant resolution from RDS PostgreSQL, tenant-specific S3 buckets, and separate Neo4j databases per tenant
> - **AppRunner Services:** Containerized services for natural language to Cypher query processing with JWT authentication and VPC connectivity
> - **Frontend Controls UI:** Complete React-based compliance controls interface with automated testing integration, comments system, evidence management, and status updates
> - **Document Editor:** Rich text document editing capabilities for compliance documentation and policy management
> - **Integration Services:** Comprehensive integration with GitHub, JIRA, and EntraID for data collection and synchronization
> - **Lambda Functions Suite:** Complete set of serverless functions for document management, integration configuration, data synchronization, and scheduled operations

---

✅ 2. Feature Breakdown & Status

Feature Name	Description	Status	Notes/details
Profile-360 Data Pipeline	Multi-tenant data collection, formatting, and Neo4j ingestion from GitHub, JIRA, EntraID	refactored profile 360 to be a part of AWS CDK and made step functions a part of AWS CDK	Step Functions orchestration, scheduled execution, error handling
Automated Control Testing	End-to-end compliance control testing with LLM analysis and evidence generation	lambda workflow that accepts with a control context performs data collections from AWS sources, data dumps and embeddings and generates compliance reports there is an old automated test for configuration,Change management in integration repo	Successfully testing TX-MON04.01, ready for generic control expansion

		https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account which for the configuration management control to check MFA in various services and check code reviews are in place in services like github	
Multi-tenant Architecture	Tenant isolation, dynamic resolution, separate databases per tenant	made profile 360 component multi-tenant in apprunner Flask API, and in AWS backend to resolve tenants from RDS	RDS tenant lookup, tenant-specific S3 buckets, Neo4j credentials from Secrets Manager
AppRunner Query Service	Apprunner is now a part of CDK and the repo is moved into AWS	added tenant resolution from RDS in apprunner stack	VPC connectivity, psycopg2 conflict resolution, LangChain .invoke() implementation
GraphQL services	Had a major Database refactor related to controls and added new GraphQL endpoints for fetching all controls, fetching control details Control tests, control test results control evidences adding, deleting and updating user comments (in controls detail page) control policies	control details page currently uses these graphql methods, some unused methods related to old controls schema in soc2_manager.py can be safely tested and removed	
Frontend Controls UI	Complete React-based compliance controls interface	UI for controls dashboard, controls detail page, comments section	Controls listing, detail pages, automated testing, comments, evidence management
Document Editor & Management	Rich text document editing for compliance	Loss of Formatting due to conversion from	Need to have a library / service that does

	documentation	binary to Document Specific Format	format conversion with 0% formatting loss
Integration Services	GitHub, JIRA, EntraID integration for data collection	Can add new orgs, of Entra, JIRA, Github, delete, sync and choose org scope (github)	Real-time data synchronization, API integrations, error handling
Policy Management Interface	Policy creation, editing, and management system	Incomplete	Bugs in ineffect-draft-template. Database inconsistencies, version control interface of policies required.
Document Lambda Function	Complete document management with S3 integration and CRUD operations	can store retrieve normal and policy type documents and can parse/convert to different file formats returing to front end	binary file to document format conversion is being done by rtf parser, there is loss of formatting, need to find a solution with 100% format preservation
List Lambda Function	List Lambda Function used to list ORGs of each integration	Can List Orgs	list orgs from s3
Configure Lambda Function	Configure Lambda Function used to configure new ORGs for integrations	Can add new orgs	can add features such as immedeate credential validation but with the limitation of API call overhead in front end (Org Status will show any failure messages)
Delete Lambda Function	Delete Lambda Function used to delete existing ORGs of integrations	Deletes specific Orgs	Delete existing org, by deleting config info and secrets
Sync Lambda Function	Sync Lambda Function used to refresh ORGs of integrations	would fetch orgs for that specific integration	sync org info serves as refresh by looking up orgs in s3 after new org addition
Scheduler Lambda Function	Scheduler Lambda Function used to schedule data dumps of integrations in S3 buckets	scheduler function currently stores clean dumps, but dirty dump is commented is available if needed	scheduler lambda hits APIs of Entra, Jira and Github to get compliance related data dumps periodically

🔐 3. Access & Credentials

Resource	Credential	Access Type	Notes
GitHub	neel-assurex	Admin	Github access
Google	neel@assurexinc.com	Admin	For Atlassian, Jira, Google workspace, Trustx platform
Entra	neel@nagarur.net	Admin	For Entra Admin console
JIRA	neel@assurexinc.com	Admin	Jira platform
AWS	neel-trustx	Dev	Orchestrates data pipeline execution
Trustx	neel@assurexinc.com (trustx) neel+trustxinc@assurexinc.com (trustxinc)	admin	Trustx platform access
Augment code	neel@assurexinc.com	admin	google account login

> All AWS resources are in us-east-2 region. PostgreSQL, Neo4j credentials are stored in AWS Secrets Manager with tenant-specific keys.

🧠 4. Technical Architecture

4.1 Database Schema

Tables Created/Used:

- controls (compliance controls public schema)
- control_implementations
- control_evidences
- control_tests
- control_test_results
- document_embeddings
- documents (document metadata and references)
- users
- comments
- policies

Schema Notes:

> Multi-tenant architecture with tenant-specific schemas (tenant_251967, tenant_876131). Each tenant has isolated data in PostgreSQL and separate Neo4j databases. Document embeddings stored with S3 keys for reference. Document content stored in S3 with metadata in PostgreSQL.

4.2 Backend - Lambda Functions

Services Used:

- AWS Lambda (Python) - Profile-360 pipeline, automated control testing, document management, integrations
- AWS Step Functions - Pipeline orchestration
- AWS AppRunner - AI query service
- PostgreSQL RDS - Tenant and compliance data
- Neo4j - Graph databases per tenant
- S3 - Document storage and results
- AWS Secrets Manager - Neo4j credentials

Document Lambda Function (manageDocuments.py):

Feature	Description	Implementation
CRUD Operations	Complete Create, Read, Update, Delete operations for documents	RESTful API with GET, POST, PUT, DELETE methods
S3 Integration	Document content storage in S3 with metadata in PostgreSQL	Automatic S3 upload/download with content type detection
Multi-format Support	Support for JSON, text, binary files, images, PDFs	Content-type based processing with base64 encoding for binary
Policy Documents	Special handling for policy document types	Policy-specific database operations and S3 storage
Version Control	Document versioning and update tracking	Timestamp-based versioning with S3 metadata
Error Handling	Comprehensive error handling and logging	Try-catch blocks with detailed error messages and rollback
Multi-tenant Support	Tenant-aware document operations	Tenant-specific S3 buckets and database schemas

Key Document Lambda Features:

- **S3 Content Storage:** Document content stored in S3 with automatic content-type detection and proper file extensions
- **Database Metadata:** Document metadata (title, description, type, etc.) stored in PostgreSQL for fast querying
- **Binary File Support:** Handles images, PDFs, and other binary files with base64 encoding for API responses
- **Policy Integration:** Special handling for policy documents with dedicated database operations
- **Content Validation:** Validates document content and handles empty or invalid content gracefully
- **Cleanup Operations:** Automatic cleanup of S3 objects when documents are deleted
- **JWT Authentication:** Secure access with JWT token validation

> Document Lambda supports multi-tenancy with automatic tenant discovery from RDS. All document content is stored in tenant-specific S3 buckets with comprehensive error handling and logging.

4.3 Frontend

Major Components Developed:

- **FocusAreaDetail.jsx** - Main control detail page with automated testing integration
- **Controls UI Components** - Complete controls listing and management interface
- **Comments System** - Comments and evidence management for controls
- **Status Update System** - Control status management and tracking
- **Document Editor** - Rich text editing for compliance documents and policies
- **Policy Management** - Policy creation, editing, and frame management
- **Integration Pages** - UI for managing GitHub, JIRA, and EntraID integrations
- **Security Headers** - CSP configuration and security policy management

Key Frontend Features:

Feature	Description	Branch/PR
Automated Control Testing	Test Control button with API integration to Lambda backend	#101 - Add automated control testing integration
Control Policies UI	Policy management interface for compliance controls	#99 - control-policies
Comments & Evidence UI	Comments section and evidence management for controls	#98 - comments and evidences ui
Control Detail Pages	Comprehensive control detail view with all metadata	#94 - control detail page
Controls UI Redesign	Modern, responsive controls interface design	#92 - controls-ui-new-design
Status Update Backend	GraphQL integration for control status management	#96 - status update backend
Document Editor	Rich text document editing with version control	document-edit-ui branch
Documents Management	Document listing, upload, and management interface	documents-v1 branch
EntraID Integration	Frontend interface for EntraID integration management	entra-id-integration branch
Policy Frames	Policy frame management and editing interface	Policy-Frames-addition branch
Policy Edit/View	Policy editing and viewing modes	Policy-Edit-View branch

Security Headers	CSP fixes for blob URLs, worker sources, CDN support	Multiple commits - headers fixes

Deployment:

> React application deployed via standard build process. Frontend connects to backend APIs for control testing functionality. Includes comprehensive security headers and CSP configuration. Document editor supports rich text editing with Material-UI components.

🚀 5. Deployment Instructions

For AWS Infrastructure:

1. cd AWS
2. npm install
3. npm run build
4. cdk deploy --all

For Lambda Stacks in AWS :

1. Deploy any stack by ex: - Profile360Stack by cd AWS && ./scripts/deploy-profile-360-stack.sh --env=dev

For Frontend:

1. cd trustx
2. npm install
3. npm run build

🛠️ 6. Local Dev Setup

1. Clone repositories: AWS, trustx, integration
2. Install dependencies: npm install in each repo
3. Configure AWS credentials for us-east-2 region
4. Set up local PostgreSQL connection for testing

🗒️ 7. Known Issues & TODOs

Area	Description	Suggested Fix / Next Steps
Document Editor	Rich text editor may need additional features	Loss of Formatting due to conversion from binary to Document Specific Format Need a solution to preserve 100% formatting
Policy Management Interface	Bugs in ineffect-draft-template. Database inconsistencies	Fix Database inconsistencies, according to the holistic definition of ineffect-draft-template functionality version control interface of policies required
Automated Testing of Controls	For automated testing if controls we have tried out various ways first through API calls https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account and through n8n we have worked on CC5.1 But now through lambdas we have a workflow that can accept any control and perform source collection and autonomous analysis Now this new workflow already	

	accepts control id, tenant-name etc. and resolves tenants and performs db querying, Right now, some prompts are hardcoded with cc7.1, but This needs to be tweaked to make it generic using the input control id also the generated documents are being stored in s3 tenant bucket in the folder automated-testing https://github.com/TrustXinc/integration/blob/main/automated-tests/automated_tests.py Connect your Github account refer this file for the existing flow to store results in relevant db tables, can be replicated exactly or can be done by giving this context to augment in one prompt altering the lambda workflow to append this save test results flow will patch the results to the front end and will reflect immediately, no other modifications needed	

📌 8. Suggested Improvements / Future Work

- **Generic Control Support:** Expand automated testing to accept any control ID and dynamically configure testing flow
- **Enhanced Data Sources:** Add additional sources and other compliance tools to data collection
- **LLM Fine-tuning:** Improve analysis quality with domain-specific training on compliance frameworks
- **Dashboard Integration:** Connect automated testing results to existing compliance dashboards
- **Parallel Processing:** Implement concurrent testing of multiple controls
- **Real-time Alerting:** Add notifications for compliance score changes
- **Performance Optimization:** Optimize data collection and processing for larger datasets

- **Frontend Enhancements:** Add real-time progress tracking, advanced filtering, and bulk operations
- **Document Collaboration:** Add real-time collaborative editing features to document editor
- **Advanced Integrations:** Expand integration capabilities to include more third-party services
- **Policy Automation:** Implement automated policy generation based on compliance requirements
- **Document Versioning:** Implement comprehensive document version control with diff capabilities
- **Bulk Document Operations:** Add support for bulk document upload, download, and management

---

📊 9. Code Contributions Summary

GitHub Statistics (neel-assurex):

- **AWS Repository:** 23+ commits including infrastructure, pipelines, and automated testing
- **trustx Repository:** 50+ commits including UI components, fixes, and API integrations
- **integration Repository:** Multiple commits for GitHub, JIRA, EntraID integration services

Major Pull Requests & Branches:

Repository	Feature/Branch	Impact
AWS	Fix automated control testing: document embeddings search and S3 storage improvements	Complete automated testing system with 32+ document embeddings
AWS	Document Lambda Stack Implementation	Complete document management system with S3 integration and CRUD operations
trustx	Add automated control testing integration to FocusAreaDetail	Frontend integration with Lambda backend for control testing
trustx	control-policies	Policy management interface for compliance controls
trustx	comments and evidences ui	Comments system and evidence management
trustx	control detail page	Comprehensive control detail views

trustx	controls-ui-new-design	Modern, responsive controls interface with Material-UI components
trustx	document-edit-ui branch	Rich text document editor with editing capabilities
trustx	documents-v1 branch	Document management system with upload and organization
trustx	integration branch	Integration interface and management
trustx	Policy-Frames-addition branch	Policy frame management and editing system
trustx	Policy-Edit-View branch	Policy editing and viewing modes

Key Technical Achievements:

- **50+ commits** across trustx repository with comprehensive UI development
- **Multiple feature branches** for document editing, policy management, and integrations
- **Material-UI integration** for modern, responsive design components
- **GraphQL client implementation** for efficient data fetching and management
- **Integration management** interfaces for external service connections
- **Scheduler Data Dumps** periodic data dumps from Entra, Jira and Github
- **Policy management system** with frame-based editing and viewing
- **Document Editor implementation** with CRUD operations and S3 integration
- **Multi-format document support** including binary files, images, and PDFs